IU Digital de Antioquia

Actividad de Aprendizaje 2 : Etapa 3 de la metodología CRISP-DM - Web Scraping

Curso : Programación para análisis de datos

Estudiante:

Juan Fernando Cataño Higuita

Óscar Javier García García

Rober Andrés Castillo Gaviria

Juan David Ramírez García

Docente:

Ana María López Moreno

Medellín

IU digital de Antioquia

17 de mayo de 2026

ÍNDICE

1. Selección de la herramienta de web scraping.....4
2. Desarrollo del Scraper.....4
3. Ingesta en la base de datos.....9
4. Pruebas y validación.....12
Link video presentación.....14
Conclusiones.....14

1. Selección de la herramienta de web scraping

El sitio web que se escogió para este trabajo fue el de la Aeronáutica Civil: aerocivil.gov.co, este dispone de una sección de consulta de vuelos mediante una url con parámetros GET que permiten filtrar por tipo de vuelo: si son de llegada o de entrada. La información mostrada se muestra mediante una tabla con la información de las aerolíneas, número de vuelo y hora de itinerario. El contenido está cargado usando una librería de JavaScript que controla la paginación del sitio web con el botón de Siguiente.

Debido a estas características, se escogió **Selenium y BeautifulSoup**. Selenium para interactuar con los controladores de paginación que usan JavaScript, permitiendo navegar por las diferentes páginas que contiene el resto de la información. Por otro lado, usamos BeautifulSoup para extraer la información HTML de las tablas, filas y columnas, las cuales son estáticas, incluyendo atributos específicos como data-fecha-archivo, para extraer la información de la fecha.

.

2. Desarrollo del Scraper

- **Para extraer la información de los vuelos de llegada usamos la siguiente url:**

https://www.aerocivil.gov.co/loader.php?lServicio=Aerolineas&lTipo=User&lFuncion=VistaUsuario&llegada=I&vuelo=&aerolinea=&origen=

- **Para extraer la información de los vuelos de salida usamos la siguiente url:**

https://www.aerocivil.gov.co/loader.php?lServicio=Aerolineas&lTipo=User&lFuncion=VistaUsuario&salida=N&vuelo=&aerolinea=&origen=

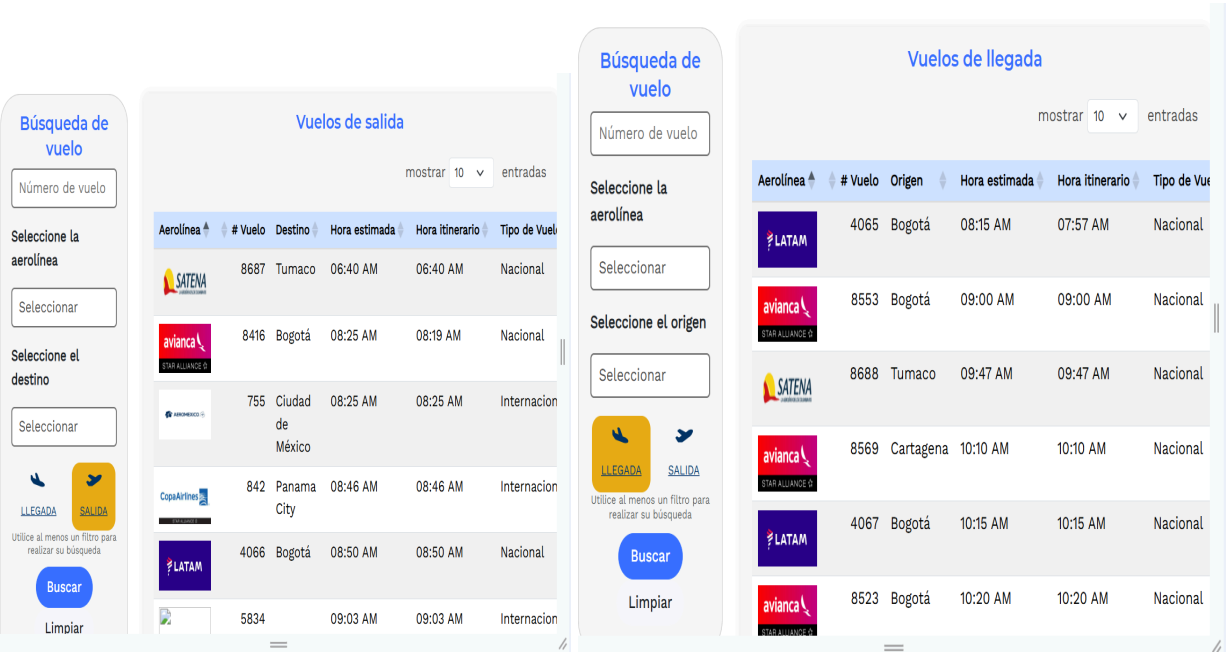
- **Enlace del video en youtube**

https://youtu.be/P7KsxPSAk_8

Repositorio para ver el script

https://github.com/JuanDavidRamirezGarcia/Programaci-n-para-el-an-lisis-de-datos-2026-1

De ambos sitios utilizamos las mismas etiquetas y selectores, ya que la estructura HTML de la página es idéntica para llegadas y salidas:



- <table> : contenedor principal de la tabla de vuelos. Se localiza con soup.find("table") para acceder a todos los datos de los vuelos.
- <tr> (table row) : cada fila de la tabla representa un vuelo individual. Se extraen con tabla.find_all("tr").
- <td> (table data) : Celdas de cada fila, accedidas con fila.find_all("td").

De ellas se extrae:

- cols[0] : Logo de la aerolínea (atributo src de la etiqueta <img>)
- cols[1] : Número de vuelo
- cols[2] : Ciudad de conexión / origen
- cols[3] : Hora estimada
- cols[5] : Tipo de vuelo (texto) y fecha del vuelo (atributo data-fecha-archivo)
- ols[6] : Estado del vuelo

- <img> : Dentro de cols[0], se accede con cols[0].find("img")["src"] para obtener la URL del logo de la aerolínea.
- Selector CSS button.page-link.next : Usado por Selenium para localizar el botón de siguiente página del componente DataTables y navegar entre páginas.

Script para obtener los vuelos de llegada. No se detallará el de los vuelos de salida porque es idéntico y se adicionará en el .zip como anexos al trabajo

1. Librerías necesarias para ejecutar el código

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from webdriver_manager.chrome import ChromeDriverManager
from bs4 import BeautifulSoup
import pandas as pd
import time
import os
```

2. URL de consulta vuelos de llegada

```
# URL específica de llegadas de la Aerocivil
URL_AEROCIVIL = "https://www.aerocivil.gov.co/loader.php?lServicio=Aerolineas&lTipo=User&lFuncion=VistaUsua
```

3. Configurador del navegador modo handle

```python
def crear_driver():
    """Crea instancia de Chrome con opciones para scraping."""
    chrome_options = Options()
    chrome_options.add_argument("--headless")  # Sin interfaz gráfica
    chrome_options.add_argument("--no-sandbox")
    chrome_options.add_argument("--disable-dev-shm-usage")
    chrome_options.add_argument("--window-size=1920,1080")
    chrome_options.add_argument(
        "user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
    )

    service = Service(ChromeDriverManager().install())
    driver = webdriver.Chrome(service=service, options=chrome_options)
    return driver
```

4. Funciones principales de extracción con paginación.

```python
def extraer_vuelos_con_paginacion(driver, max_paginas=10):
    """Navega por todas las páginas disponibles y extrae los vuelos."""
    todos_los_vuelos = []
    pagina_actual = 1

    print(f"Navegando a: {URL_AEROCIVIL}")
    driver.get(URL_AEROCIVIL)

    while pagina_actual <= max_paginas:
        print(f" Procesando página {pagina_actual}...")

        # 1. Esperar a que la tabla de la página actual esté lista
        try:
            WebDriverWait(driver, 10).until(
                EC.presence_of_element_located((By.TAG_NAME, "table"))
            )
        except:
            print("No se pudo cargar la tabla.")
            break
```

Espera y carga de la tabla HTML

```python
# 2. Extraer datos de la página actual con BeautifulSoup
soup = BeautifulSoup(driver.page_source, "lxml")
tabla = soup.find("table")
if tabla:
    filas = tabla.find_all("tr")
    for fila in filas:
        cols = fila.find_all("td")
        if len(cols) >= 5:
            vuelo = {
                "Aerolínea": cols[0].find("img")["src"].strip() if cols[0].find("img") else "",
                "Numero_Vuelo": cols[1].get_text(strip=True),
                "Ciudad_Conexion": cols[2].get_text(strip=True),
                "Hora_Estimada": cols[3].get_text(strip=True),
                "Fecha_Vuelo": cols[5].get("data-fecha-archivo", "").strip(),
                "Tipo_de_Vuelo": cols[5].get_text(strip=True),
                "Estado": cols[6].get_text(strip=True),
                "Tipo_movimiento": "Llegada",
                "Pagina_Fuente": pagina_actual
            }
            if vuelo["Aerolínea"].lower() != "aerolínea":
                todos_los_vuelos.append(vuelo)
```

Extracción de campos con BeautifulSoup y navegación a la siguiente página

```python
    # 3. Intentar ir a la siguiente página
    try:
        # Buscamos el botón por su clase 'next' y que no esté deshabilitado
        boton_siguiente = driver.find_element(By.CSS_SELECTOR, "button.page-link.next")

        # Verificamos si el botón está deshabilitado (DataTables suele añadir la clase 'disabled')
        if "disabled" in boton_siguiente.get_attribute("class") or not boton_siguiente.is_enabled():
            print("Llegamos al final de la lista.")
            break

        # Hacer clic usando JavaScript para evitar errores de elementos ocultos o superpuestos
        driver.execute_script("arguments[0].click();", boton_siguiente)

        # Pausa para que cargue la nueva página de datos
        time.sleep(2)
        pagina_actual += 1

    except Exception as e:
        # Si no encuentra el botón, significa que no hay más páginas
        print("No hay más páginas disponibles.")
        break

return todos_los_vuelos
```

5. Ejecución principal, guardado del CSV y vista previa

```python
def main():
    print("=" * 60)
    print("  SCRAPER AEROCIVIL - ESTADO DE VUELOS DE LLEGADA")
    print("=" * 60)

    driver = crear_driver()

    try:
        # Extraer datos
        datos_vuelos = extraer_vuelos_con_paginacion(driver, max_paginas=10)

        if not datos_vuelos:
            print("No se obtuvieron datos.")
            return

        # Paso 4: Guardar en CSV con Pandas
        df = pd.DataFrame(datos_vuelos)

        # Crear carpeta de salida
        if not os.path.exists("output"):
            os.makedirs("output")

        archivo_csv = "output/vuelos_llegadas_selenium.csv"
        df.to_csv(archivo_csv, index=False, encoding="utf-8-sig")

        print(f"\n{'=' * 60}")
        print(f"  Datos guardados en: {archivo_csv}")
        print(f"  Total de vuelos capturados: {len(df)}")
        print(f"{'=' * 60}")

        # Vista previa en consola
        print("\nVista previa de los primeros vuelos:")
        print(df.head(10).to_string(index=False))

    finally:
        driver.quit()
        print("\nNavegador cerrado.")
```

3. Ingesta en la base de datos

Una vez obtenidos los archivos CSV generados por los scrapers de llegadas y salidas, este script se encarga de ejecutar el proceso ETL (Extract, Transform, Load). Primero carga y limpia los datos desde los archivos CSV, luego aplica transformaciones sobre los campos necesarios como la normalización del nombre de la aerolínea, y finalmente sube la información a una base de datos SQL Server Express, distribuyéndola en dos tablas de staging independientes (Staging_Llegada y Staging_Salida) y una tabla de hechos unificada (Fact_Vuelos) que consolida tanto los vuelos de llegada como los de salida.

1. Librerías necesarias para ejecutar el código

```
import pandas as pd
from sqlalchemy import create_engine, text
```

2. Configuración de rutas de archivos CSV y conexión a la base de datos

```
# Configuración de conexión
file_path_1= r'output\vuelos_llegadas_selenium.csv'
file_path_2= r'output\vuelos_salidas_selenium.csv'
server = r'LAPTOP-5G1JGRV9\SQLEXPRESS'
database = 'EstadosVuelosDB'

engine = create_engine(f'mssql+pyodbc://{server}/{database}?driver=ODBC+Driver+17+for+SQL+Server')
```

3. carga de archivos CSV y limpieza de datos

```
def load_and_upload():

    print("Cargando datos desde CSV...")
    # Cargamos y limpiamos espacios en los nombres de las columnas
    df1= pd.read_csv(file_path_1)
    df2= pd.read_csv(file_path_2)
    df1.columns = df1.columns.str.strip()
    df2.columns = df2.columns.str.strip()
```

Creación de tablas Staging para almacenar los datos de salida y llegada para posteriormente ser limpiados los datos.

```
create_tables_query = """
DROP TABLE IF EXISTS Staging_LLegada;
DROP TABLE IF EXISTS Staging_Salida;
DROP TABLE IF EXISTS Fact_Vuelos;


CREATE TABLE Staging_LLegada (
    id_llegada INT PRIMARY KEY IDENTITY(1,1),
    aerolinea VARCHAR(255),
    numero_vuelo INT,
    ciudad_conexion VARCHAR(255),
    hora_estimada VARCHAR(50),
    fecha_vuelo VARCHAR(50),
    tipo_de_vuelo VARCHAR(50),
    estado VARCHAR(255),
    tipo_movimiento VARCHAR(50),
    pagina_fuente VARCHAR(255)
);

CREATE TABLE Staging_Salida (
    id_salida INT PRIMARY KEY IDENTITY(1,1),
    aerolinea VARCHAR(255),
    numero_vuelo INT,
    ciudad_conexion VARCHAR(255),
    hora_estimada VARCHAR(50),
    fecha_vuelo VARCHAR(50),
    tipo_de_vuelo VARCHAR(50),
    estado VARCHAR(255),
    tipo_movimiento VARCHAR(50),
    pagina_fuente VARCHAR(255)
);
"""

with engine.connect() as connection:
    print("Limpiando y creando tablas...")
    connection.execute(text(create_tables_query))
    connection.commit()
```

4. Selección de columnas, eliminación de duplicados y limpieza del campo aerolínea

```
# --- CREACIÓN DE DIMENSIONES ---

# Llegada: BORRAMOS DUPLICADOS PARA ASEGURAR QUE NUMERO_VUELO SEA ÚNICO
llegada_df = df1[[ 'Aerolínea','Numero_Vuelo', 'Ciudad_Conexion', 'Hora_Estimada', 'Fecha_Vuelo', 'Tipo_de_Vuelo', 'Estado', 'Tipo_movimiento']].drop
llegada_df.columns = ['aerolinea', 'numero_vuelo', 'ciudad_conexion', 'hora_estimada', 'fecha_vuelo', 'tipo_de_vuelo', 'estado', 'tipo_movimiento']

# Salida: BORRAMOS DUPLICADOS PARA ASEGURAR QUE NUMERO_VUELO SEA ÚNICO
salida_df = df2[['Aerolínea','Numero_Vuelo', 'Ciudad_Conexion', 'Hora_Estimada', 'Fecha_Vuelo', 'Tipo_de_Vuelo', 'Estado', 'Tipo_movimiento']].drop_d
salida_df.columns = ['aerolinea', 'numero_vuelo', 'ciudad_conexion', 'hora_estimada', 'fecha_vuelo', 'tipo_de_vuelo', 'estado', 'tipo_movimiento']


# Limpiar el campo aerolinea para extraer solo el nombre
def limpiar_aerolinea(df):
    df["aerolinea"] = (df["aerolinea"]
        .str.split('/').str[-1]       # toma el nombre del archivo
        .str.split('_').str[-1]       # quita el prefijo numérico
        .str.replace(r'\.(jpg|jpeg|png)$', '', regex=True)  # quita extensión
        .str.title()                  # capitaliza → "Avianca", "Latam"
    )
    return df

llegada_df = limpiar_aerolinea(llegada_df)
salida_df = limpiar_aerolinea(salida_df)
```

5. Carga de datos a las tablas de staging en SQL Server

```
#---Carga de datos a staging---

print("Cargando dimensiones de staging a SQL...")

llegada_df.to_sql('Staging_LLegada', engine, if_exists='append', index=False)
salida_df.to_sql('Staging_Salida', engine, if_exists='append', index=False)
```

6. Carga y consolidación de la tabla hechos.

```
# Combinar ambas tablas (Verticalmente)
df_hechos_vuelos = pd.concat([llegada_df, salida_df], ignore_index=True)

# Cargar a SQL
df_hechos_vuelos.to_sql('Fact_Vuelos', engine, if_exists='append', index=False
print("Carga completa. Datos insertados en Fact_Vuelos.")

f __name__ == "__main__":
    load_and_upload()
```

4. Pruebas y validación

Muestra de datos cargados correctamente a la base de datos:

Los vuelos de llegada y salidas fueron almacenados en una sola tabla fact_vuelos para un mejor manejo de la información en un único espacio.

```
use EstadosVuelosDB;
Go


select * from Fact_vuelos;
```

154 %

Results | Messages

	aerolinea	numero_vuelo	ciudad_conexion	hora_estimada	fecha_vuelo	tipo_de_vuelo	estado	tipo_movimiento
1	Latam	4063	Bogotá	07:15 AM	2026-05-09 07:43	Nacional	NULL	Llegada
2	Avianca	8493	Bogotá	07:40 AM	2026-05-09 07:43	Nacional	NULL	Llegada
3	Jetsmart	5302	Bogotá	07:45 AM	2026-05-09 07:43	Nacional	NULL	Llegada
4	Latam	4065	Bogotá	08:15 AM	2026-05-09 07:43	Nacional	NULL	Llegada
5	Satena	8688	Tumaco	08:52 AM	2026-05-09 07:43	Nacional	NULL	Llegada
6	Avianca	8553	Bogotá	09:00 AM	2026-05-09 07:43	Nacional	NULL	Llegada
7	Avianca	8465	Bogotá	09:30 AM	2026-05-09 07:43	Nacional	NULL	Llegada
8	Jetsmart	5593	San Andrés	09:38 AM	2026-05-09 07:43	Nacional	NULL	Llegada
9	Latam	4067	Bogotá	10:05 AM	2026-05-09 07:43	Nacional	NULL	Llegada
10	Jetsmart	5532	Cartagena	10:16 AM	2026-05-09 07:43	Nacional	NULL	Llegada
11	Avianca	8523	Bogotá	10:20 AM	2026-05-09 07:43	Nacional	NULL	Llegada
12	Clic	6989	Pasto	10:35 AM	2026-05-09 07:43	Nacional	NULL	Llegada
13	Satena	6545	Bahía Solano	10:52 AM	2026-05-09 07:43	Nacional	NULL	Llegada
14	Arilines	282	Panama City	11:06 AM	2026-05-09 07:43	Internacional	NULL	Llegada
15	Satena	8653	Guapi	11:21 AM	2026-05-09 07:43	Nacional	NULL	Llegada
16	Clic	7795	Tumaco	11:25 AM	2026-05-09 07:43	Nacional	NULL	Llegada
17	Satena	6549	Nuquí	11:47 AM	2026-05-09 07:43	Nacional	NULL	Llegada
18	Avianca	8567	Bogotá	12:00 PM	2026-05-09 07:43	Nacional	NULL	Llegada
19	Avianca	9227	Bogotá	12:25 PM	2026-05-09 07:43	Nacional	NULL	Llegada
20	Clic	6909	Guapi	12:45 PM	2026-05-09 07:43	Nacional	NULL	Llegada
21	Latam	4075	Bogotá	12:55 PM	2026-05-09 07:43	Nacional	NULL	Llegada
22	Aerolineadefecto	810	NULL	01:08 PM	2026-05-09 07:43	Internacional	NULL	Llegada
23	American	919	Miami	01:08 PM	2026-05-09 07:43	Internacional	NULL	Llegada
24	Avianca	8411	Bogotá	01:15 PM	2026-05-09 07:43	Nacional	NULL	Llegada
25	Satena	7745	Pitalito	01:55 PM	2026-05-09 07:43	Nacional	NULL	Llegada

Manejo de errores

Ambos scrapers implementan bloques try/except en dos puntos clave: la espera de carga de la tabla *HTML* y la navegación entre páginas, interpretando la ausencia del botón de siguiente página como el fin de la paginación. El bloque *try/finally* en *main()* garantiza que el navegador se cierre correctamente ante cualquier error. En el ETL, el bloque with *engine.connect()* asegura que la conexión a SQL Server se libere correctamente al finalizar.

Reporte técnico

Entre las decisiones tomadas fue combinar Selenium con BeautifulSoup ya que inicialmente se consideró usar solo BeautifulSoup, pero al identificar que la tabla de vuelos usa DataTables para la paginación del lado del cliente, usar Selenium se volvió necesario para navegar entre las diferentes páginas de la tabla. BeautifulSoup se mantuvo para el parseo del HTML por su simplicidad y eficiencia.

Para el ETL se eligió SQL Server Express por disponibilidad local, usando SQLAlchemy como conector. Se diseñaron dos tablas de staging independientes para llegadas y salidas, y una tabla de hechos unificada Fact_Vuelos para facilitar consultas consolidadas.

Desafíos enfrentados

El primer desafío fue el campo de aerolínea ya que el sitio entrega la URL de imagen en lugar de texto. Se resolvió en el ETL extrayendo el nombre desde el nombre del archivo con operaciones de string sobre el DataFrame.

El segundo fue el manejo de la paginación: el botón de siguiente página requería un clic vía execute_script() en lugar del clic directo de Selenium, ya que elementos superpuestos del sitio bloqueaban la interacción normal.

Finalmente, al renombrar las columnas del DataFrame antes de aplicar la limpieza del campo aerolínea, se generó un KeyError por usar el nombre original Aerolínea en lugar del nuevo aerolinea. Se corrigió reordenando las operaciones en el ETL.

Link video presentación

https://youtu.be/P7KsxPSAk_8

Conclusiones

- La combinación de Selenium y BeautifulSoup demostró ser la estrategia más adecuada para abordar las características técnicas del portal de la Aeronáutica Civil, permitiendo superar la limitación de la paginación dinámica gestionada por DataTables y garantizar la extracción completa de los vuelos de llegada y salida.
- Se implementó exitosamente el proceso ETL, logrando transformar y cargar los datos extraídos en una base de datos SQL Server estructurada en tablas de staging independientes

y una tabla de hechos unificada Fact_Vuelos, que consolida la información de ambos tipos de movimiento para su posterior análisis.

- La identificación y resolución de los desafíos encontrados durante el desarrollo, como la normalización del campo de aerolínea y el manejo de la paginación, evidenció la importancia de comprender a fondo la estructura del sitio web antes de iniciar el proceso de scraping, optimizando así la calidad y consistencia de los datos obtenidos.